

LLM 预训练之 Attention

Self-Attention / MHA / GQA / MLA / FlashAttention / Sparse Attention

- 深度研究手册

从 Q/K/V 的内容寻址，到 GPU IO-aware 内核，再到可训练稀疏路由：Attention 如何决定“信息从哪里来、以什么带宽流向哪里”

核心视角：Attention = 内容相关的信息路由器 + 二次复杂度系统瓶颈

Research cut-off: 2026-08-13 | 01-B-Agent

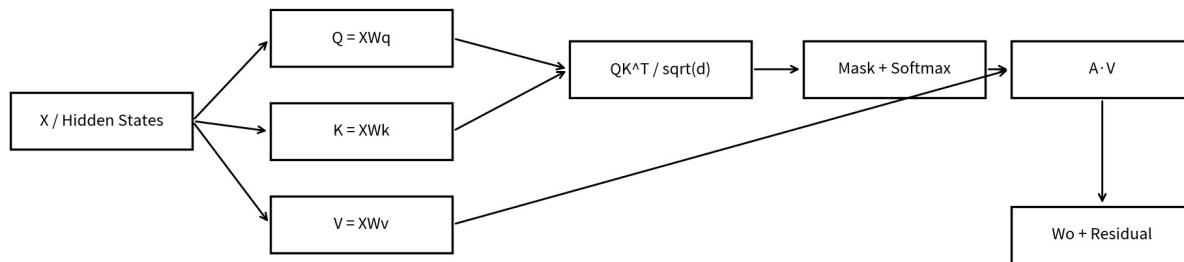
执行摘要：Attention 的两条主线必须同时理解

Attention 的“算法线”回答：模型如何根据当前 query，在历史 token 中动态选择信息；Attention 的“系统线”回答：这个动态选择怎样在 GPU 上以可承受的显存、带宽和通信成本完成。现代 LLM 的 Attention 设计，几乎都可以理解成对这两条主线之一或同时优化。

核心判断	Know-why	工程含义
Q/K 决定“读哪里”，V 决定“读什么”	Attention 是内容相关路由，而不是固定卷积核	分析 head 时应区分路由几何与 value 内容
Softmax attention 的关键瓶颈是 L^2	每个 query 原则上要与所有 key 比较	长上下文必须优化内核、稀疏性或近似方式
MHA 的多头是表示子空间并行	不同 heads 可学习不同路由模式	但并非所有 head 都同样重要，存在冗余
GQA/MQA 主要压缩 KV，而非 Q	decode 时 KV cache / memory bandwidth 是大成本	训练结构会直接影响 serving economics
FlashAttention 不改变数学定义	减少 HBM↔SRAM IO，避免 materialize $L \times L$ 矩阵	“算法 FLOPs 不变”仍可能大幅 wall-clock 加速
Sparse attention 不是一种方法	fixed pattern、learned routing、block routing 的 inductive bias 完全不同	必须用长任务 + 硬件吞吐同时评测
有效 Attention 不等于可视化好看	attention weight 只是中间路由变量之一	解释 head 需要 intervention / ablation，而非只看热力图

最重要的 Know-why Attention 之所以成为 Transformer 的核心，不是因为它“记住了所有 token”，而是因为它让每一层、每个位置都能根据当前内容动态建立一张临时通信图。

Scaled Dot-Product Self-Attention：信息路由的数据流



Attention 的核心不是“平均上下文”，而是：用 Q/K 计算内容相关的路由权重，再对 V 做加权读取。

图 1 Self-Attention 的信息流：Q/K 产生路由权重，V 携带被读取的内容。

1. Scaled Dot-Product Attention：最核心的一行公式

Transformer 把 attention 定义为 $\text{Attention}(Q,K,V)=\text{softmax}(QK^T/\sqrt{d_k})V$ ；Multi-Head Attention 则在多个投影子空间中并行执行该过程，再拼接并投影。原论文的关键创新之一，是用全并行 self-attention 取代 recurrence/convolution，从而显著提高训练并行性。[1]

对于 decoder-only LLM，还需要 causal mask：位置 t 只能看到 $\leq t$ 的 key/value。训练时整段序列仍可并行计算；推理时则逐 token 增量生成。

组件	数学角色	工程直觉	常见故障
$Q = XW_q$	当前 token 的检索向量	“我现在需要什么？”	Q norm / logit 过大导致 softmax 过尖
$K = XW_k$	历史 token 的索引向量	“我能被怎样匹配？”	KV 维度与缓存成本直接相关
$V = XW_v$	被读取的内容向量	“如果选中我，拿走什么？”	value scale 过大可放大 residual outlier
$QK^T/\sqrt{d}$	相似度 logits	构造临时路由图	L^2 计算/显存，数值范围随维度增长
Softmax	归一化权重	竞争式选择 / mixture	饱和、attention sink、无关上下文分配概率质量
Mask	可见性约束	定义信息能否跨位置传播	packing boundary 错误会造成跨文档污染

2. 为什么要除以 $\sqrt{d_k}$ ：不是装饰项

若 q 、 k 各维近似零均值、单位方差，则 $q \cdot k$ 的方差随 d_k 增长。维度越大，未缩放 logits 越容易进入 softmax 饱和区，梯度变小。除以 $\sqrt{d_k}$ 是把 logit scale 拉回更稳定范围。

QK-Norm 进一步把 query/key 先做 L^2 normalization，并使用可学习 scale，目标就是降低 softmax 被任意大范数推入饱和区的风险；其原始实验来自低资源翻译，但这个“控制 attention logit geometry”的思想后来被更广泛采用。
[12]

工程观察 训练大模型时，attention logit mean/max、Q/K RMS、softmax entropy、attention output norm 都值得监控；只看 global loss 很难及时发现 attention 子系统逐层饱和。

3. Multi-Head Attention：多头究竟提供了什么

单头 Attention 在一个投影空间中构造一种路由。多头把 hidden state 投到多个较小子空间，让每个 head 同时建立不同的匹配函数，再通过 output projection 混合。它的价值不是“看得更远”，而是并行表达不同关系。

早期 head 分析研究发现，部分 heads 会形成稳定且可解释的角色；但大量 heads 又可以在测试时被剪掉而性能下降有限。Michel 等发现某些层甚至可缩减到单 head；Voita 等在英俄翻译中剪掉 48 个 encoder heads 中的 38 个，BLEU 只下降 0.15。[18][19]

现象	说明什么	不要误解成
head specialization	某些 heads 学到位置、指代、语法或复制类模式	每个 head 都有清晰“语义名称”
head redundancy	模型过参数化，多个 heads 可承担近似功能	多头完全没有必要
layer dependence	浅层/深层 heads 的功能与冗余程度不同	所有层应使用同样 head 数
training dynamics	多头可改善优化路径，即使最终可剪	“能剪掉” = “训练时不需要”

Know-how 如果要研究 head importance，应做 head masking / causal intervention / retraining ablation；单纯看 attention heatmap 很容易把“相关性”误当成“因果作用”。

4. Attention 的成本模型：为什么 L^2 是系统级问题

对每层 self-attention， QK^T 与 AV 都含有与 L^2 成比例的工作量；朴素实现还会 materialize $L \times L$ attention matrix。序列从 8K 增加到 128K，是 $16 \times$ 长度，却对应约 $256 \times$ 的 dense pairwise interaction 数。

为什么长上下文会逼迫 Attention 改算法或改内核

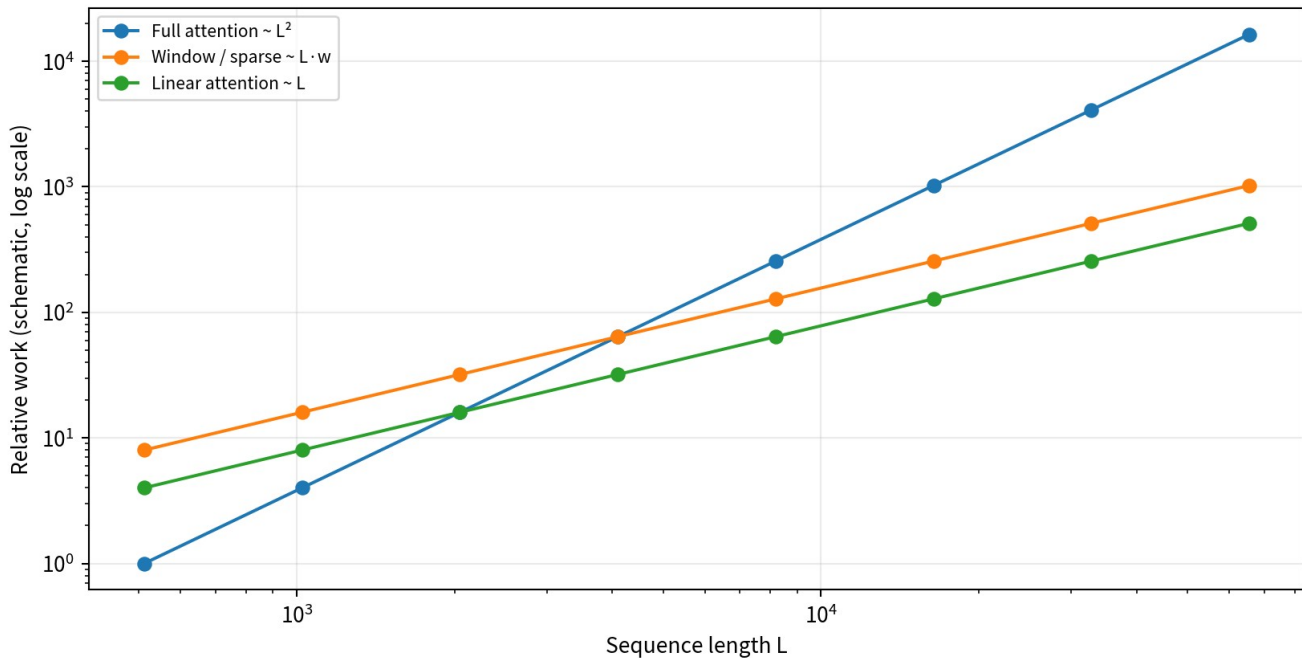


图2 Attention 复杂度的增长方向示意：full dense 的 L^2 是长上下文最核心的压力源。

成本	训练阶段	推理 Prefill	推理 Decode
QK / AV FLOPs	高，随 L^2 增长	高，随 prompt 长度 ²	每步约随历史长度 L 增长

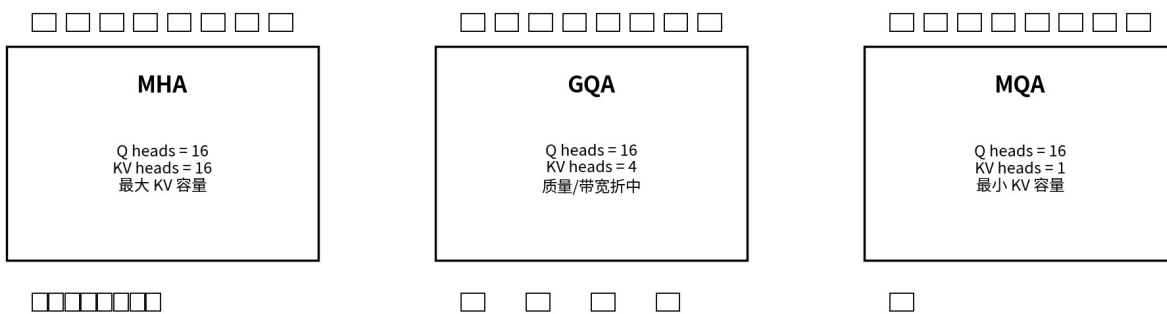
成本	训练阶段	推理 Prefill	推理 Decode
Attention matrix memory	朴素实现高	朴素实现高	通常不长期保存完整矩阵
KV cache	训练通常不作为持久缓存	生成 K/V	核心内存与带宽瓶颈
Kernel launch / IO	影响 wall-clock	影响首 token 延迟	memory bandwidth 往往主导

5. MHA → MQA → GQA：为什么把 Q 头与 KV 头解耦

Multi-Query Attention (MQA) 保留多个 query heads，但所有 heads 共享同一组 K/V。Shazeer 提出该设计的直接动机是自回归 decode 时反复读取大量 K/V tensors 的 memory-bandwidth 成本，并报告可显著加速解码、质量仅有小幅下降。[2]

Grouped-Query Attention (GQA) 则在 MHA 与 MQA 之间取中间点：多个 Q heads 被分组，每组共享 K/V。Ainslie 等报告可用约原预训练 5% compute 把 MHA checkpoint uptrain 成 MQA/GQA，且 GQA 获得接近 MHA 的质量与接近 MQA 的速度。[3]

MHA → GQA → MQA：Query 多头与 KV 多头可以解耦



对自回归推理，KV cache 大小近似正比于 n_{kv_heads} ；GQA/MQA 的主要目标是降低 memory bandwidth，而不是改变 Q head 数。

图 3 GQA/MQA 的本质是减少 KV heads；这直接缩小 KV cache 和 decode 时读取的 K/V 字节数。

粗略地，单层 KV cache bytes $\propto 2 \times L \times n_{kv_heads} \times d_{head} \times bytes_per_element$ 。于是 n_{kv_heads} 从 32 降到 8，KV cache 理论上约降为 1/4；这也是为什么今天的 decoder LLM 常把 n_q_heads 与 n_{kv_heads} 分开配置。

6. MLA：比 GQA 更进一步的“KV 潜变量压缩”

DeepSeek-V2 引入 Multi-head Latent Attention (MLA)，核心不是仅减少 KV head 数，而是先把 key/value 信息压缩进低维 latent，再从该 latent 产生各 heads 所需的表示。其技术报告把显著压缩 KV cache 作为主要贡献之一，并报告相对 DeepSeek 67B 减少 93.3% KV cache、提升最大生成吞吐。[7]

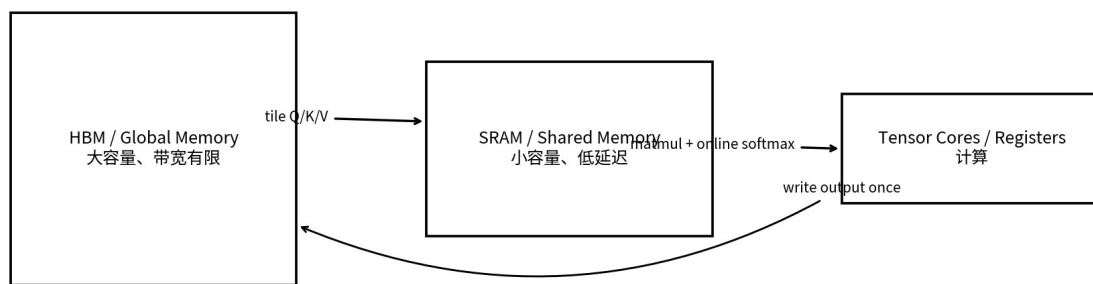
方法	共享/压缩位置	优势	代价/注意
MHA	不共享 KV heads	最大独立表达	KV cache 最大
GQA	每组共享 K/V	质量/带宽折中成熟	仍保存多组 K/V
MQA	所有 Q 共享单组 K/V	KV 最小、实现直接	潜在质量损失
MLA	先压缩到 latent，再恢复/组合	更激进 KV 压缩	结构更复杂，与位置编码/解码实现耦合更强

Know-why GQA/MQA 在“head 维度”压缩 KV；MLA 在“表示维度”压缩 KV。两者解决同一个系统问题，但压缩自由度不同。

7. FlashAttention：为什么“不近似”也能大幅加速

FlashAttention 的关键论点是：标准 attention 的瓶颈不能只按 FLOPs 看，必须考虑 GPU memory hierarchy。它通过 tiling 在 SRAM 中分块计算，并使用 online softmax，避免把完整 attention matrix 写到 HBM；算法仍是 exact attention。原论文报告在 GPT-2 1K sequence 上约 3× 训练加速，并把 attention memory 从二次量级的显式矩阵降为线性工作空间。[4]

FlashAttention 的 IO-aware 视角：算术复杂度不变，内存访问模式改变



FlashAttention 的关键不是减少 $O(L^2)$ 算术量，而是避免把完整 $L \times L$ attention matrix 反复写回 HBM。

图4 FlashAttention 的关键优化对象是 HBM↔SRAM 数据搬运，而不是改变 softmax attention 的数学定义。

FlashAttention-2 继续优化 thread-block / warp work partition，减少非 matmul FLOPs，并在 A100 上达到约 50%-73% 理论峰值 FLOPs；其 GPT-style 训练实验最高约 225 TFLOPs/s/A100、72% MFU。[5]

FlashAttention-3 面向 Hopper 的异步 Tensor Core/TMA 与 FP8，在 H100 上报告 FP16 最多约 740 TFLOPs/s、FP8 接近 1.2 PFLOPs/s，并通过更好的低精度处理降低数值误差。[6]

版本	主要瓶颈认知	核心改动	意义
FlashAttention	HBM IO 太多	tiling + online softmax	exact attention 首次把 IO 复杂度放到中心
FlashAttention-2	GPU work partition 不充分	更好并行、减少 non-matmul FLOPs	让 attention 更接近 GEMM 效率
FlashAttention-3	Hopper 新硬件能力未利用	异步、warp specialization、FP8	attention kernel 与硬件代际深度共设计

一个经常被混淆的点 FlashAttention 主要解决“dense exact attention 怎么更快”；Sparse/Linear Attention 解决“是否还要计算所有 pair”。两者不是同一层优化，而且可以组合。

8. Attention Mask：它其实定义了信息图

Causal mask、padding mask、document-boundary mask、sliding-window mask、global-token mask，本质上都不是数据预处理细节，而是在定义 token 之间允许存在的有向边。Sequence Construction 中一个 boundary mask 的错误，会直接把原本独立文档连成一张错误的信息图。

Mask 类型	允许的边	典型用途
Causal	$j \leq i$	decoder LM
Padding	排除 padding token	variable length batch
Intra-document	同文档内 causal	packed documents
Sliding Window	局部窗口内	长上下文低成本
Global Tokens	少量 token $\leftrightarrow$ 全序列	Longformer/BigBird 类设计
Block Sparse	部分 blocks 间连接	硬件友好的稀疏 attention

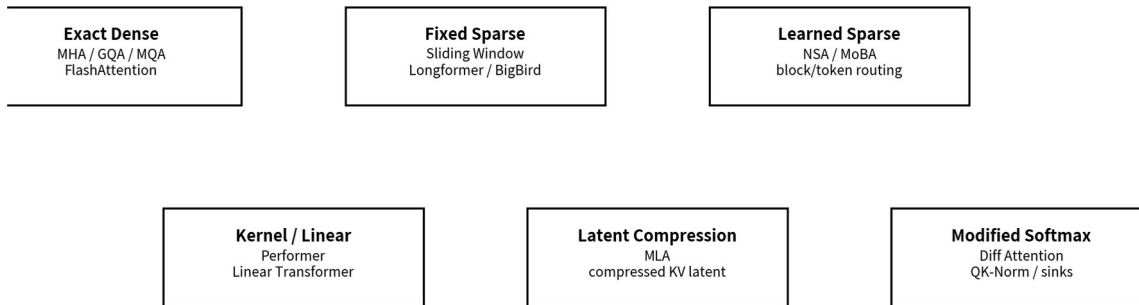
9. Sparse Attention：固定模式、动态路由与原生可训练

Longformer 使用 local window + task-motivated global attention，使 attention 随序列长度线性扩展；BigBird 使用局部、随机与全局连接，并给出其稀疏结构在理论表达能力上的结果。两者代表“人为设计固定稀疏图”的经典路线。[14][15]

2025 年 DeepSeek 的 Native Sparse Attention (NSA) 把稀疏策略做成原生可训练的层级结构：粗粒度压缩 + 细粒度 token selection，并强调从训练 forward/backward 到 decode 的硬件对齐；论文报告在 64K 序列上相对 Full Attention 有显著加速，同时一般/长上下文/推理 benchmarks 持平或更好。[8]

同年 MoBA 把 MoE 的路由思想移到 Attention：query 根据相关性选择一小部分 KV blocks，并允许 full/sparse attention 之间平滑过渡。其目标是减少强人工结构偏置，让模型自己决定该看哪些 block。[9]

Attention 方法地图：不要把“高效 Attention”视为同一种技术



设计空间的本质：保持 full softmax 语义、减少 KV、减少被访问的 token/block、近似 kernel，或改变归一化/打分本身。

图5 “高效 Attention”至少包含六类不同设计，不能只按 complexity 标签归为一类。

路线	稀疏由谁决定	Complexity 直觉	主要风险
Sliding Window	人工固定窗口	$O(Lw)$	远距离信息必须靠层间传播或 global tokens
BigBird-like	固定 + 随机 + global	近似 $O(L)$	pattern 与任务不匹配
NSA	模型/算法联合选择	减少实际被访问 KV	router/selection 本身要高效且可训练
MoBA	query 到 block 的 learned routing	block-level sparse	block size / routing SNR 与 GPU kernel 强耦合

2025-2026 趋势 Sparse Attention 正从“设计一个漂亮的稀疏图”转向“让稀疏路由可训练、可解释地保留重要上下文，并从第一天就为 GPU block/kernel 形状设计”。

10. Linear / Kernel Attention：为什么理论上 $O(L)$ ，但没有全面替代 softmax

Linear Transformer 利用核特征映射与矩阵乘法结合律，把某些 attention 形式改写为线性复杂度，并揭示其可递归计算的 RNN-like 形式。Performer 则用 FAVOR+ 随机特征近似 softmax attention kernel，给出线性空间/时间与近似误差保证。[17][16]

优点	关键限制
长序列计算与状态可近似线性	通常改变/近似标准 softmax attention 的表达形式
自回归可递归更新固定大小状态	精确的“任意 query 对任意历史 token 的检索”能力可能变弱
理论分析漂亮，内存优势明显	实际 wall-clock 依赖 kernel、硬件与任务；复杂 reasoning 的质量 trade-off 必须实测

Know-why Full attention 的昂贵之处恰恰来自它没有预先决定“该忘什么”；线性状态压缩必须把任意长历史浓缩进固定统计量，因此天然存在信息选择压力。

11. Ring Attention：不是减少 L^2 ，而是把长序列分布到设备

Ring Attention 把 sequence 分块分布到多设备，并让 K/V blocks 沿 ring 传递，同时将通信与 blockwise attention 计算重叠。它不近似 full attention，而是通过 sequence parallelism 把可训练 context length 扩展到随设备数增长。[10]

这类方法提醒我们：Attention 的“算法复杂度”和“单卡可容纳长度”不是同一个问题。即使仍做 full attention，只要把激活/sequence 分片并重叠通信，也能突破单设备显存边界。

12. Attention Sink：Softmax 为什么会把概率质量倒向“无意义位置”

StreamingLLM 观察到，滑动窗口只保留最近 K/V 时性能会崩，但额外保留序列开头少量 token 的 K/V 能显著恢复质量；作者把这种对初始 token 的强注意称为 attention sink，并发现预训练时加入 dedicated sink token 可进一步改善 streaming。[11]

Attention sink 的工程含义很重要：softmax 强制每个 query 的 attention weights 总和为 1，即使“没有值得看的历史 token”，概率质量仍必须分配到某处。某些 heads 因此可能学出稳定的默认汇点。

2026 年的新理论工作进一步证明，在一类 trigger-conditional 任务中，softmax self-attention 为实现“默认忽略输入”的行为会必然产生 sink；这属于较新的理论结果，适合作为机制假说的一部分，而不应当解释所有架构中的 sink 来源。[20]

现象	可能机制	工程对策
BOS/首 token 高 attention	默认路由 / softmax 质量出口	保留 sink KV；显式 sink token；修改 attention 形式
远距离 token 被系统性忽略	局部偏置、位置/训练分布、softmax 竞争	长上下文训练、routing/sparse design、位置标定
无关上下文吸走 attention	logit noise + 全局竞争	Diff Attention、QK norm、retrieval-aware sparse attention

13. Differential Attention：把两个 softmax 图相减

Differential Transformer 用两个独立 softmax attention maps 的差值来抵消共同噪声，目标是加强相关上下文、削弱无关上下文。论文在语言模型 scaling、长上下文检索、ICL 与 hallucination 相关任务中报告了优势。[13]

这条路线代表一个更基础的研究方向：不只压缩或稀疏标准 softmax attention，而是重新思考“attention weights 必须是一个单一概率分布吗？”

14. 预训练阶段真正该监控哪些 Attention 指标

层级	指标	为什么看
Numerics	Q/K RMS、attention logit max/min、NaN/Inf	提前发现 softmax 饱和或数值崩坏
Distribution	attention entropy、top-k mass、sink mass	判断路由是否过尖、过平或集中到固定位置
Geometry	QK cosine / norm 分布、head correlation	发现 heads 退化/冗余/过度共享
Output	attention output RMS、residual contribution	区分“权重很尖”与“实际输出很大”
Head utilization	head ablation sensitivity、gradient norm	识别 dead/redundant/specialized heads
Long context	distance-conditioned attention、retrieval accuracy vs position	验证模型真的在使用远距离信息
System	TFLOPs/s、HBM bandwidth、kernel occupancy、attention time %	区分算法问题与 kernel/IO 问题

Know-how Attention 指标最好与 layer index、head index、sequence length、data domain 联合切片。一个全局平

均 entropy 几乎总会掩盖真正的问题。

15. Attention Ablation：如何判断新机制真的更好

必须固定	为什么
参数量 / active params	避免用更大模型“买”到收益
训练 token / data mixture / tokenizer	Attention 对长文档比例和 packing 特别敏感
global batch / LR / optimizer	不同 attention 可能改变稳定区间
sequence length distribution	稀疏 attention 的收益常只在长序列出现
kernel maturity	算法好但 kernel 差，会错误得出 wall-clock 结论
KV precision / cache policy	推理结果否则无法公平比较

最低限度应同时报告：train loss / val loss、短上下文 benchmark、长上下文 retrieval+reasoning、tokens/s、MFU、peak memory、prefill latency、decode throughput、KV bytes/token。只报告 FLOPs 或只报告 benchmark 都是不完整的。

16. Production Blueprint：预训练 Attention 的工程分层

层	责任	典型选择
Semantics	定义能看谁、如何归一化	causal mask / softmax / relative position interaction
Projection	Q/K/V 参数化与 head sharing	MHA / GQA / MQA / MLA
Sparsity	是否计算所有 pair	full / window / block sparse / learned routing
Kernel	同一数学语义怎样跑快	SDPA / FlashAttention-2/3 / custom sparse kernels
Parallelism	跨设备怎样切	tensor / sequence / context / ring attention
Numerics	控制 logits 与低精度误差	$\sqrt{d}$ scale / QK norm / FP8 strategy
Observability	证明 attention 在工作	entropy / sink / head usage / distance profile
Evaluation	验证能力与系统收益	short + long + prefill + decode + memory

推荐的默认路径 如果没有强证据需要改语义：优先 GQA + exact softmax + 成熟 FlashAttention 内核；只有当目标 context/throughput 使 dense attention 成为不可接受的核心成本时，再进入 NSA/MoBA/linear 等结构性改变，并通过从头预训练 ablation 验证。

17. 方法选择决策表

场景	优先候选	原因
通用 decoder LLM, $\leq 32K$	GQA + FlashAttention	成熟、质量风险低
长 prompt, decode KV 成本高	更少 KV heads / MLA	直接降低 cache 与带宽
64K-1M 训练, 仍要 full semantics	FlashAttention + context/ring parallel	不改 attention 语义
超长上下文且 dense FLOPs 不可接受	NSA / MoBA 类 learned sparse	让模型学习选择少量相关区域
流式无限历史	sliding window + sink-aware design	限制缓存同时保持稳定
极端线性复杂度需求	kernel/linear attention	接受表达形式改变, 需强任务验证
训练 attention logit 不稳定	QK norm / scale diagnostics	先修数值几何, 不要直接换稀疏结构

18. Know-Why / Know-How 总结

Know-why	Know-how
Attention 是动态通信图, 不是“记忆表”	把 Q/K 路由、V 内容、mask 可见性分开诊断
L^2 是 pairwise interaction 的代价	先用 IO-aware exact kernel, 再判断是否必须结构性稀疏
KV cache 决定 decode economics	通过 GQA/MQA/MLA 控制 n_{kv_heads} 与表示维度
多头既有 specialization 也有 redundancy	用 intervention/ablation 评估, 而不是看热力图讲故事
长上下文有算法、显存、通信三种瓶颈	区分 FlashAttention、Sparse Attention、Ring/Context Parallel 的层级
Softmax 自带竞争与归一化偏置	监控 entropy、sink、logit scale; 必要时研究 QK norm / modified attention
硬件实现是 Attention 设计的一部分	性能比较必须绑定 GPU、dtype、sequence length 与 kernel 版本

最终心智模型 Q/K = 地址系统；V = 内容载荷；Softmax = 竞争式路由协议；Mask = 网络拓扑；Heads = 并行子网络；FlashAttention = 路由计算内核；GQA/MLA = 缓存压缩；Sparse Attention = 动态或固定地减少网络边。

参考资料与证据等级

- [1] Vaswani et al. (2017), Attention Is All You Need. [\[奠基论文 / 一手\] source](#)
- [2] Shazeer (2019), Fast Transformer Decoding: One Write-Head is All You Need. [\[MQA / 一手\] source](#)
- [3] Ainslie et al. (2023), GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. [\[GQA / 一手\] source](#)
- [4] Dao et al. (2022), FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. [\[系统算法 / 一手\] source](#)
- [5] Dao (2023), FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. [\[系统算法 / 一手\] source](#)
- [6] Shah et al. (2024), FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. [\[系统算法 / 一手\] source](#)
- [7] DeepSeek-AI (2024), DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. [\[MLA 技术报告 / 一手\] source](#)
- [8] Yuan et al. (2025), Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. [\[Sparse attention / 一手\] source](#)
- [9] Lu et al. (2025), MoBA: Mixture of Block Attention for Long-Context LLMs. [\[Block routing / 一手\] source](#)
- [10] Liu et al. (2023), Ring Attention with Blockwise Transformers for Near-Infinite Context. [\[Context parallel / 一手\] source](#)
- [11] Xiao et al. (2023), Efficient Streaming Language Models with Attention Sinks. [\[Attention sink / 一手\] source](#)
- [12] Henry et al. (2020), Query-Key Normalization for Transformers. [\[Numerics / 一手\] source](#)
- [13] Ye et al. (2024), Differential Transformer. [\[Modified attention / 一手\] source](#)
- [14] Beltagy et al. (2020), Longformer: The Long-Document Transformer. [\[Fixed sparse / 一手\] source](#)
- [15] Zaheer et al. (2020), Big Bird: Transformers for Longer Sequences. [\[Fixed sparse / 一手\] source](#)
- [16] Choromanski et al. (2020), Rethinking Attention with Performers. [\[Kernel attention / 一手\] source](#)
- [17] Katharopoulos et al. (2020), Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. [\[Linear attention / 一手\] source](#)
- [18] Michel et al. (2019), Are Sixteen Heads Really Better than One? [\[Head pruning / 一手\] source](#)
- [19] Voita et al. (2019), Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting. [\[Head analysis / 一手\] source](#)
- [20] Ran-Milo (2026), Attention Sinks Are Provably Necessary in Softmax Transformers. [\[前沿理论 / 需持续复现\] source](#)

说明：本手册中的“机制解释/工程建议”是基于上述一手论文的综合分析；论文中给出的速度、困惑度、内存或 benchmark 数字均属于各自实验设置，不应脱离硬件、模型规模、数据和实现版本直接横向复用。